#1 We have to use an explicit stack to simulate the behavior of recursive calls (see next section)
#2 Pushing a node on the stack is the equivalent to a recursive call in iterative terms

The two methods are equivalent, but look at how elegant is the first one. Try to write both versions for post-order binary tree traversal, and you'll also realize that the iterative one is harder to get right.

E.1.3 Tail Recursion

Every time we make a function call, we create a new entry in the program stack, a so-called *stack frame*. The stack frame consists of a few fields that are needed by our program or virtual machine to resume execute the called function and then, when it's done, resume execution of the caller function.

A (non-exhaustive) list of these fields includes:

- The *program counter* (a pointer to the next instruction in the caller function to run after the new function finishes);
- All the arguments passed to the called function;
- All the local variables;
- A placeholder for the return value of the called function.

Recursive calls are no exception. With the caveat that a recursive function might not return before the base case is hit: this means the call chain can get extremely long.